Chiwire Maps 1: Device Setup

For the owner, before the first trip. This guide takes a LilyGo T-Dongle S3 with a GPS to a working offline map of mainland Spain: firmware, SD card, map tiles, search data and the first start. The commands are for Linux (tested on Ubuntu); notes for Windows and macOS are given where they differ, but those were not tested.

How long it takes, measured on the build machine: about 42 hours of unattended tile download, 2 hours of copying to the card, and about an hour of your own time.

Contents

1. What the device is, and what it can't do
2. Installing the tools
3. Changing the Wi-Fi name and password
4. Building and flashing the firmware
5. Formatting the SD card
6. What goes on the card
7. Downloading the map tiles
8. Building the search data
9. Copying to the SD card
10. First start checklist

What the device is, and what it can't do

Chiwire Maps is a small USB stick computer (an ESP32-S3) with a screen, a GPS and a microSD card. It makes its own Wi-Fi network. Any phone that joins the network opens the map in its web browser: no app, no mobile data and no internet are needed.

The dongle only stores and sends files. Everything else (drawing the map, the grid, the search, the norths, the sun times) runs in the phone's browser. That keeps the dongle within its small memory.

Limits to know before you rely on it:

Limit	**What it means for you**
About 160–170 KB of free memory	The dongle answers "Low Memory" (HTTP 503) below 40 KB; the page retries on its own
Slow Wi-Fi (hundreds of KB/s, shared)	Map tiles appear one by one; the vector layer is the slowest
At most 4 phones at once	The ESP32's default limit for its own network

Limit	What it means for you
Plain `http://`, no HTTPS	Phones block the compass sensor on such pages; the north arrows come from models, not from the phone
FAT32 card	Files up to 4 GB each; the card must use 4 KB clusters (see below)
64 GB card	About 59.6 GB usable; the default download uses 54 GB
Magnetic model WMM2025	Valid until the end of 2029; replace it then (guide 5)
Mainland Spain only	No maps of the Balearic or Canary Islands, Ceuta or Melilla

The device helps you navigate. It doesn't replace a paper map, a compass and the skill to use them (guide 3).

Installing the tools

You need: a PC with Linux, a USB-C cable for the dongle, a microSD card reader, and about 70 GB of free disk space for the tiles.

1. **Install VS Code and PlatformIO.** Install VS Code, open the Extensions panel, search for "PlatformIO IDE" and install it. PlatformIO downloads the ESP32 compiler the first time it builds. Its command-line tool is then at `~/.platformio/penv/bin/pio`. It is not on your PATH, so this guide always writes the full path.
2. **Install Python 3.** It is already installed on most Linux systems. Check with `python3 --version`. The tile downloader needs only Python's standard library.
3. **Allow your user to use the USB serial port.** Run these two commands, then log out and log back in (or restart):

   ```
   sudo usermod -aG dialout $USER
   curl -fsSL
   https://raw.githubusercontent.com/platformio/platformio-core/develop/
   platformio/assets/system/99-platformio-udev.rules | sudo tee
   /etc/udev/rules.d/99-platformio-udev.rules
   sudo udevadm control --reload-rules && sudo udevadm trigger
   ```

   Check afterwards: `groups` must list `dialout`. With the dongle plugged in, `ls -l /dev/ttyACM*` must show `/dev/ttyACM0`.
4. **Get the project.** Clone or download this repository. The commands below run from its root folder unless they say otherwise.

Changing the Wi-Fi name and password

The network name and password are in `firmware/src/main.cpp`:

```
const char* ssid = "chiwire";
const char* password = "aguacate"; // Set to "" for no password
```

Change both before you build your own device. A WPA2 password must have at least 8 characters. The QR code on the dongle's screen is made from these values, so it updates on its own.

Building and flashing the firmware

1. **Build.** From the `firmware/` folder, run:

   ```
   cd firmware
   ~/.platformio/penv/bin/pio run
   ```

   The first build downloads the libraries and takes a few minutes. It ends with `[SUCCESS]` and the memory use (about RAM 14.0%, Flash 29.9%).

2. **Put the dongle in flash mode.** Unplug it. Hold the BOOT button, plug it in, then release the button. The screen stays dark; that is normal in flash mode.

3. **Upload.** Run:

   ```
   ~/.platformio/penv/bin/pio run -t upload --upload-port /dev/ttyACM0
   ```

   Expect a percentage counter and "Hard resetting via RTS pin…" at the end.

4. **Restart normally.** Unplug the dongle and plug it in again **without** holding the button. The new firmware starts.

5. **Optional: read the log.** `~/.platformio/penv/bin/pio device monitor` shows what the dongle prints: the SD card check, the network address, the GPS statistics every 10 seconds. Press Ctrl+C to leave. The monitor and the upload can't use the port at the same time.

Formatting the SD card

The card must be **FAT32 with 4 KB clusters**:

- **Why FAT32 and not exFAT:** the ESP32 framework this firmware is built with cannot read exFAT. An exFAT card is simply not recognised (the LED turns red).
- **Why 4 KB clusters:** the map is 3.66 million small files. Each file takes at least one whole cluster. With the default 32 KB clusters, the same tiles would need 118 GB instead of 54 GB.
- **FAT32 limit:** no single file can be larger than 4 GB.

Formatting erases the whole card. On Linux:

1. Insert the card and find its name: `lsblk -o NAME,SIZE,LABEL`. A 64 GB card shows as about 59.6G, for example `sdb` with the partition `sdb1`. The letter changes between computers

and even between insertions; check it every time.

2. Unmount it if the desktop mounted it: `sudo umount /dev/sdX1` (replace `sdX1`).
3. Format the partition with 8 sectors of 512 bytes per cluster (4 KB) and the label BEACON:

```
sudo mkfs.vfat -F 32 -s 8 -S 512 -n BEACON /dev/sdX1
```

On Windows, the built-in tools offer FAT32 only up to 32 GB; use a third-party FAT32 formatter that lets you choose a 4096-byte allocation unit. On macOS, `newfs_msdos -F 32 -c 8 -v BEACON /dev/rdiskNs1` gives 4 KB clusters (find `diskN` with `diskutil list`). Neither was tested for this project.

What goes on the card

The repository's `sdCard/` folder is the card's template. After the downloads below it holds everything:

```
/                       the map app: nav.js, geo.js, marks.js, search.js, wmm.js,
                        leaflet.js, leaflet.css, favicon.ico, icon-192.png,
error.png
/lib/                   MapLibre, for the vector layer
/vector/                the vector map style, fonts and icons
/tiles/<layer>/z/x/y    map tiles: mtn, pnoa, base (.jpg) and vector (.pbf)
/search/                the search data (*.json.gz)
/serve/                 your own files, shown on the Files page (guides, routes,
PDFs)
```

The map app lives on the card, not in the firmware, so it can be updated by copying files. Only the two page shells (`firmware/include/index.h` and `files.h`) need a reflash when they change.

Downloading the map tiles

The tiles come from the Spanish National Geographic Institute (IGN), under the CC BY 4.0 license. The plan in `tileDownloader/plan_spain.json` downloads:

- the whole peninsula to zoom 14 in all four layers (MTN topographic, PNOA aerial, IGN Base streets, vector);
- the mountain ranges (Pyrenees, Cantabrian Mountains and Picos de Europa, Sistema Central, the northern and southern Iberian System, Sierra Nevada, Cazorla-Segura) and the Community of Madrid to zoom 16, in the three raster layers.

Measured result: 3,663,112 tiles, **54 GB** on a card with 4 KB clusters.

Layer	Tiles	Average tile	On the card
MTN (topographic)	1,155,601	14.5 KB	18.2 GB
PNOA (aerial photo)	1,155,601	14.1 KB	17.8 GB
IGN Base (streets)	1,155,601	6.4 KB	~9.2 GB
Vector (zoom ≤ 14)	196,309	~39 KB	~7.7 GB

Steps, from the `tileDownloader/` folder:

1. **See the size and time first.** This downloads nothing:

```
cd tileDownloader
python3 spain_tiles.py estimate --rate 24
```

2. **Start the download.** Run it under `systemd-inhibit`, so the laptop doesn't go to sleep and freeze it. Keep the laptop plugged in, open and ventilated:

```
systemd-inhibit --what=sleep:idle:handle-lid-switch --who="ChiwireMaps tiles"
--why="Downloading Spain map tiles" \
  python3 spain_tiles.py download --out ../sdCard/tiles --adaptive --rate 24
--max-rate 24 --yes
```

   At 24 tiles per second the full plan takes about **42 hours**. The default speed is 5 tiles per second (about 204 hours).

3. **If it stops, run the same command again.** It skips the tiles already on disk and continues. Ctrl+C stops it cleanly within seconds. Only one download can run per folder.

4. **Fetch the broken tiles.** A few tiles fail even after 4 attempts (IGN answers "502 Bad Gateway" now and then, about 0.01% of tiles). They are listed in `../sdCard/tiles/failed.txt`. Afterwards, run:

```
python3 spain_tiles.py retry --out ../sdCard/tiles --yes --rate 10
```

5. **Download the vector style, fonts and icons** (small, needed for the vector layer):

```
python3 spain_tiles.py vector --out ../sdCard/vector
```

Being polite to IGN. IGN publishes no speed limit. The adaptive mode (`--adaptive`) watches the error rate: more than 1% failed requests in a 2-minute window halves the speed. Every speed from 5 to 25 tiles per second ran clean in the measured download. Don't use proxies or several computers to go faster.

Short of space or time? Dropping the IGN Base layer at zoom 15–16 (the second step in `plan_spain.json`) saves about 7.3 GB and 53 hours of download at 5 tiles/s. In the mountains, MTN already shows the paths. `plan_madrid_test.json` is a small test plan of central Madrid: add `--plan plan_madrid_test.json` to try everything in minutes.

Building the search data

The search (streets with house numbers, towns, metro, peaks, shelters, water…) uses OpenStreetMap data. It is built on the PC and stored on the card.

1. **Make a Python environment with pyosmium** (once):

```
python3 -m venv tools/.venv
tools/.venv/bin/pip install osmium==4.3.1
```

2. **Download the Spain extract** from Geofabrik (about 1.5 GB) into `tools/data/`:

```
mkdir -p tools/data
curl -L -o tools/data/spain-latest.osm.pbf
https://download.geofabrik.de/europe/spain-latest.osm.pbf
```

3. **Build it:**

```
tools/.venv/bin/python tools/build_search.py tools/data/spain-latest.osm.pbf
sdCard/search
```

   Measured: 5 minutes 17 seconds, with a peak of 7.4 GB of RAM. The output is 166 MB in 19,492 files, with 2.02 million house numbers. Only mainland Spain is kept, like the maps.

The data is © OpenStreetMap contributors, under the ODbL 1.0 license.

Copying to the SD card

Millions of small files make this slow, and one desktop setting makes it much slower. The desktop mounts FAT32 cards with the `flush` option, which waits for the card after every file: measured 8 files per second, about 5 days for this card. Mount the card by hand without it: measured about 350 files per second, **2 hours 5 minutes** for the full card.

1. **Find the card:** `lsblk -o NAME,SIZE,LABEL` (label BEACON). If the desktop mounted it, unmount it first: `sudo umount /dev/sdX1`.
2. **Mount it by hand:**

```
sudo mkdir -p /mnt/beacon
sudo mount -t vfat -o uid=1000,gid=1000,utf8,shortname=mixed,noatime
/dev/sdX1 /mnt/beacon
```

   If you pulled the card out earlier while it was mounted, run `sudo umount -l /mnt/beacon` first.

3. **Copy everything:**

```
rsync -rt --modify-window=1 --info=progress2 --exclude='.download.lock' \
  --exclude='download_metrics*.csv' --exclude='failed.txt' sdCard/
/mnt/beacon/
```

   `-rt` copies folders and times but not Linux permissions, which FAT32 can't store. `--modify-window=1` is needed because FAT32 stores times in 2-second steps; without it, a second run copies everything again. Running the same command again resumes an interrupted copy.

4. **Follow the progress** in a second terminal: `python3 tools/card_copy_progress.py`. rsync's own percentage is meaningless for millions of files.
5. **Copy the guides** (optional, so they are on the device): `mkdir -p /mnt/beacon/serve/guides && cp docs/guides/*.pdf`

`/mnt/beacon/serve/guides/`

6. **Unmount before removing the card:** `sudo umount /mnt/beacon`. Linux keeps up to a few GB in memory ahead of the card; the unmount waits until everything is written. It can take minutes.

Updating only the app later. After a change to the map app, copy all its files, not only the changed ones:

```
cp sdCard/{*.js,favicon.ico,icon-192.png} /mnt/beacon/ && sync
```

A stale file on the card (for example an old `nav.js`) hides newer features.

Checking a full copy (optional, slow): the rsync command with `-n --out-format='%n'` lists every file that differs; an empty list means the card is complete. It took 50 minutes on this card.

First start checklist

1. Insert the card into the dongle **before** plugging it in.
2. Plug the dongle into a USB charger or power bank. The capybara appears for a moment, and the LED turns yellow while it starts.
3. The LED goes off (no phone yet) or green (GPS position found). Red means the card was not found: check that it is FAT32 and fully inserted.
4. The screen shows "YOU ARE OFF GRID" and a QR code. Scan it with the phone camera to join the Wi-Fi network.
5. Open `http://192.168.4.1` in the phone's browser. The map opens over Madrid until the GPS finds a position. Outdoors this can take a few minutes the first time.
6. Check each layer with the 🗺️ button, the 🔍 search and the 🗂️ Files page. Guide 2 explains everything on the screen.